

# React.js

## Day 3 — React Router, Programmatic Navigation & Protected Routes

*Student Notes · Beginner Friendly · Interview Ready*

React Router | BrowserRouter | Routes & Route | Link | useNavigate | Protected Routes | Context API  
Auth

## What We Learned Today

- React Router — navigating between pages
- SPA vs Traditional Websites
- BrowserRouter, Routes, Route, Link
- Programmatic Navigation
- useNavigate Hook
- Protected Routes
- Authentication using Context API

### 1. What is Routing?

Moving between pages inside a React application

*Routing means navigating from one page to another inside an application — without loading a brand new website each time.*

### Example Route Flow

Home → About → Services → Contact

In React, only the required component is swapped — the browser does not reload.

### 2. Traditional Website vs React SPA

Understanding the core difference in how pages work

#### Traditional Website

- Reloads the entire page on every navigation
- Reloads the complete DOM
- Header and Navbar load again and again
- Slower performance and longer wait times
- Each page has its own HTML file

#### React SPA (Single Page Application)

- Only one HTML file — index.html
- Only the required component is swapped
- No full page reload at all
- Faster, smoother user experience
- Header and Navbar stay mounted

**Note:** SPA = Single Page Application. React follows the SPA approach by default.

### 3. What is React Router?

The library that powers navigation in React apps

*React Router is a library that enables navigation between pages in React applications — without refreshing the browser.*

### Install React Router

```
npm install react-router-dom
```

**Note:** The package name is `react-router-dom` — not `react-router`. Always check the full name.

### 4. Core Components of React Router

The four building blocks you use in every project

| Component     | Role                               | Key Point                                                            |
|---------------|------------------------------------|----------------------------------------------------------------------|
| BrowserRouter | Wraps the entire application       | Enables routing — tracks browser URL — must be the outermost wrapper |
| Routes        | Container for all Route components | Checks the current URL and displays the matching page                |
| Route         | Maps a URL path to a component     | <code>path="/about"</code> + <code>element={&lt;About /&gt;}</code>  |
| Link          | Navigation element for users       | Replaces HTML <code>&lt;a&gt;</code> tag — does not reload the page  |

### Code Example — Basic Setup

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      {/* Navigation links */}
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/contact">Contact</Link>
      </nav>
    </BrowserRouter>
  );
}
```

```

    { /* Page routes */
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  </BrowserRouter>
);
}

```

## Link vs HTML <a> Tag

| Link (React Router)                 | <a> tag (HTML)                     |
|-------------------------------------|------------------------------------|
| Does NOT reload the page            | Reloads the entire page            |
| Changes only the required component | Destroys and rebuilds the full DOM |
| Fast SPA navigation                 | Traditional website navigation     |
| <Link to="/about">About</Link>      | <a href="/about">About</a>         |

✗ Never use <a href=...> for internal navigation in React — always use <Link>

✓ Use <Link to=...> for all internal page navigation in React Router

## Routing Flow — How it Works

```

    User clicks a <Link>
      ↓
    URL changes in the browser address bar
      ↓
    <Routes> checks the current URL
      ↓
    Matching <Route> is found
      ↓
    That Route's component renders on screen
  
```

## 5. Programmatic Navigation

Navigating using JavaScript logic instead of Links

*Programmatic Navigation means moving between pages using JavaScript code — not by clicking a Link component.*

### When Do You Use It?

| Situation             | Navigate To       |
|-----------------------|-------------------|
| Login successful      | Dashboard         |
| Form submitted        | Success page      |
| Logout clicked        | Login page        |
| Product added to cart | Cart page         |
| Payment completed     | Confirmation page |

## 6. useNavigate Hook

The React Router hook for JavaScript-driven navigation

*useNavigate() is a React Router hook that returns a navigate function. Call that function with a path to move to another page using JavaScript.*

### Syntax

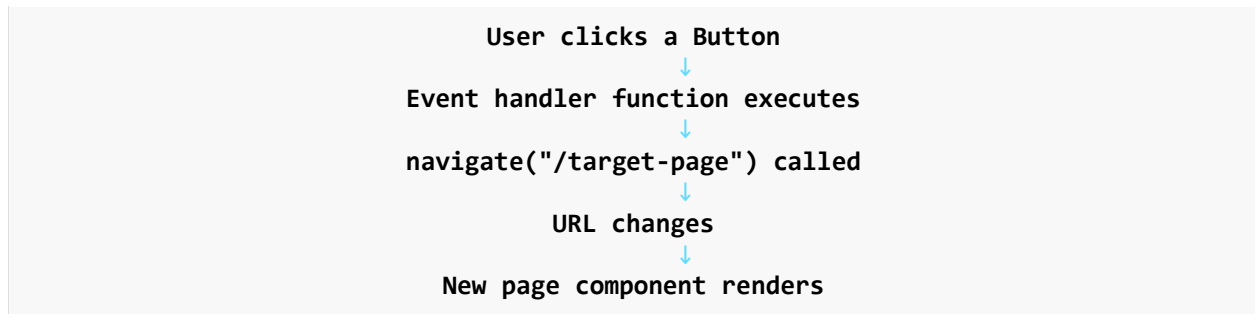
```
import { useNavigate } from 'react-router-dom';

function LoginPage() {
  const navigate = useNavigate();

  function handleLogin() {
    // ... verify credentials ...
    navigate("/dashboard"); // programmatic redirect
  }

  return <button onClick={handleLogin}>Login</button>;
}
```

## Programmatic Navigation Flow



## Key Points

- `useNavigate()` must be called inside a React component — not outside
- The `navigate()` function accepts a path string as argument
- Use `navigate(-1)` to go back — like browser back button
- The component must be inside `BrowserRouter` for the hook to work

## 7. Protected Routes

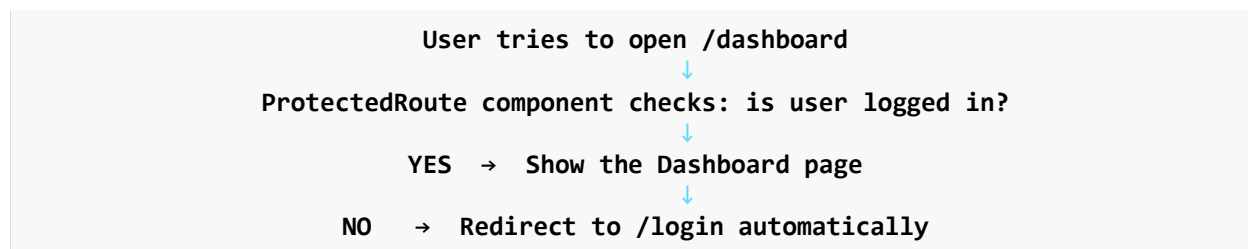
Restricting access to pages based on login status

*Protected Routes are routes that only logged-in users can access. If a user is not logged in, they are automatically redirected to the Login page.*

### Pages That Need Protection

| Protected Page | Why?                                   |
|----------------|----------------------------------------|
| Dashboard      | Shows personal or sensitive user data  |
| Profile        | Contains private account details       |
| Admin Panel    | Only administrators should access this |
| Settings       | User-specific configurations           |
| Cart / Orders  | Linked to a specific user's account    |

### Authentication Flow



## 8. AuthContext — Storing Login State

Using Context API to share auth state across all components

*AuthContext stores the login status globally so every component in the app can know whether the user is logged in — without prop drilling.*

### What AuthContext Provides

- `isLogin` — boolean: true if logged in, false if not
- `login()` — function: sets `isLogin` to true
- `logout()` — function: sets `isLogin` to false

### AuthContext Code Structure

```
import { createContext, useState, useContext } from 'react';

// 1. Create the context
const AuthContext = createContext();

// 2. Create a provider component
export function AuthProvider({ children }) {
  const [isLogin, setIsLogin] = useState(false);

  function login() { setIsLogin(true); }
  function logout() { setIsLogin(false); }

  return (
    <AuthContext.Provider value={{ isLogin, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// 3. Custom hook to use auth anywhere
export function useAuth() {
  return useContext(AuthContext);
}
```



## Wrap Your App with AuthProvider

```
// main.jsx
root.render(
  <AuthProvider>
    <App />
  </AuthProvider>
);
```

## ProtectedRoute Component

*ProtectedRoute is a wrapper component that checks login status and either shows the page or redirects to Login.*

```
import { Navigate } from 'react-router-dom';
import { useAuth } from './AuthContext';

function ProtectedRoute({ children }) {
  const { isLoggedIn } = useAuth();

  if (!isLoggedIn) {
    return <Navigate to="/login" />; // redirect if not logged in
  }

  return children; // show the page if logged in
}
```

## Using ProtectedRoute in App.jsx

```
<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/dashboard" element={
    <ProtectedRoute>
      <Dashboard />
    </ProtectedRoute>
  } />
</Routes>
```

## Login & Logout Flow

### Login Flow

```
graph TD; A[User enters username and clicks Login] --> B[credentials verified]; B --> C[login() function called - isLogin becomes true]; C --> D[navigate("/dashboard") called]; D --> E[Dashboard renders (ProtectedRoute passes)];
```

### Logout Flow

```
graph TD; A[User clicks Logout button] --> B[logout() function called - isLogin becomes false]; B --> C[navigate("/login") called]; C --> D[Login page renders];
```

---

## Real-Life Analogy — Company Office

| React Concept  | Real-Life Equivalent | Behaviour                                     |
|----------------|----------------------|-----------------------------------------------|
| Login Page     | Reception desk       | First place every visitor arrives             |
| AuthContext    | Employee database    | Stores who is authenticated                   |
| ProtectedRoute | Security guard       | Checks credentials before letting you through |
| Dashboard      | Employee-only office | Only accessible to authorized users           |
| useNavigate    | Security escort      | Takes you to the right place after checking   |

**Note:** Without permission, the security guard (ProtectedRoute) sends you back to reception (Login page).

## Real-World Applications

- Banking applications — protect account and transaction pages
  - E-commerce websites — protect cart, orders, and checkout pages
  - Student portals — protect grades and attendance pages
  - Admin dashboards — protect data management panels
  - Employee management systems — protect HR data
  - Social media platforms — protect profile and messaging pages
- 

## Common Beginner Mistakes

- ✗ Forgetting to install react-router-dom before using it
  - ✗ Not wrapping the app with BrowserRouter — routes won't work at all
  - ✗ Using `<a href=...>` instead of `<Link to=...>` — causes full page reload
  - ✗ Wrong path names — /About vs /about (paths are case-sensitive)
  - ✗ Forgetting to import the page components in App.jsx
  - ✗ Using `useNavigate()` outside a React component — causes errors
  - ✗ Thinking Protected Routes provide complete security — they only protect the UI
- 

## Key Points to Remember

| Concept                    | One-Line Summary                                                                |
|----------------------------|---------------------------------------------------------------------------------|
| SPA                        | Single HTML file — React swaps components, no reload                            |
| BrowserRouter              | Wraps the app — enables URL tracking                                            |
| Routes                     | Container that checks URL and renders matching Route                            |
| Route                      | Maps path= to element= (the component to show)                                  |
| Link                       | Navigation without reload — always use instead of <code>&lt;a&gt;</code>        |
| <code>useNavigate()</code> | Hook for JavaScript-driven navigation (after login, form submit etc.)           |
| Protected Route            | Wrapper that checks <code>isLogin</code> — redirect if false, show page if true |
| AuthContext                | Context API storage for login state — accessible app-wide                       |
| Frontend security          | Protected Routes improve UX — backend auth is still required                    |

## Interview Questions & Answers

### **Q: What is React Router?**

**A:** A library that enables navigation between pages in a React app without refreshing the browser.

### **Q: Why do we use React Router?**

**A:** To create Single Page Applications with smooth, fast navigation and no full page reloads.

### **Q: What is BrowserRouter?**

**A:** It wraps the entire React app, enables routing, and tracks the browser URL.

### **Q: Difference between Routes and Route?**

**A:** Routes is a container that holds all routes and checks the URL. Route maps a specific URL path to a specific component.

### **Q: Why use <Link> instead of <a>?**

**A:** <Link> changes pages without refreshing the browser. <a> causes a full page reload, which defeats the purpose of SPA.

### **Q: What is Programmatic Navigation?**

**A:** Navigation that happens through JavaScript code instead of clicking Link components — for example after login or form submission.

### **Q: What is useNavigate?**

**A:** A React Router hook that returns a navigate function to move between pages programmatically.

### **Q: When is useNavigate commonly used?**

**A:** After login, logout, form submission, payment success, or any action that should redirect the user automatically.

### **Q: What are Protected Routes?**

**A:** Routes that only logged-in users can access. If not logged in, the user is redirected to the Login page.

**Q: Are Protected Routes enough for security?**

**A:** No. They only protect the frontend UI. The backend must also verify authentication and authorization for each API request.

**Q: What is AuthContext?**

**A:** A Context API object that stores and shares login state globally — so every component can check if the user is logged in.

---

## Practice Tasks

### Task 1 — Multi-Page Website

- Create pages: Home, About, Services, Contact
- Use BrowserRouter, Routes, Route, Link
- Build a Navbar with Link components

### Task 2 — Login to Dashboard Redirect

- Create a Login page with a username input and login button
- On login click, navigate to /dashboard using useNavigate()

### Task 3 — Protected Dashboard

- Dashboard page should only be accessible when logged in
- If not logged in, redirect to Login page automatically
- Use ProtectedRoute component and AuthContext

### Task 4 — Logout Flow

- Add a Logout button on the Dashboard
- On click, call logout() and navigate('/login')
- Verify that /dashboard is no longer accessible after logout

## Task 5 — Custom Navigation Bar

- Build a Navbar component with Link to all pages
  - Show Login button if user is not logged in
  - Show Logout button if user is logged in (use isLogin from AuthContext)
- 

## Key Takeaways

- ✓ React follows the SPA approach — one HTML page, components swap on navigation.
- ✓ React Router enables client-side routing — install with `npm install react-router-dom`.
- ✓ BrowserRouter wraps the whole app — must be the outermost wrapper.
- ✓ Routes holds multiple Route components. Route maps path to element.
- ✓ Always use `<Link>` for internal navigation — never `<a>` tags inside React Router.
- ✓ `useNavigate()` provides programmatic navigation — use inside components only.
- ✓ Protected Routes check `isLogin` and redirect to Login if the user is not authenticated.
- ✓ AuthContext provides login state globally using Context API — no prop drilling needed.
- ✓ Frontend Protected Routes improve UX but are NOT a security replacement for backend auth.